

AIN SHAMS UNIVERSITY

Faculty of Engineering

Computer and Artificial Intelligence Engineering

MicroGrad

Scalar Automatic Differentiation Engine

Deep Learning – Assignment 1

Prepared by

Abdelrahman Adel Fathy

Student ID: 23p0144

Technical implementation, mathematical derivations, and validation

November 2026

1 Abstract and objectives

This assignment implements a small *scalar reverse-mode automatic differentiation* system from scratch. The `Value` class builds a directed acyclic computational graph while evaluating arithmetic expressions. Each operation stores its local derivative rule; a reverse topological traversal propagates gradients using the chain rule. The engine powers a fully connected neural network API comprising `Neuron`, `Layer`, and `MLP`. No PyTorch or TensorFlow is used to compute gradients or train the model.

2 Computational graph and differentiation

2.1 Data representation and operator overloading

Each `Value` stores scalar `data`, gradient `grad`, a set of parents `_prev`, operation metadata `_op`, an optional label, and a local `_backward` callback. Arithmetic creates a new output node connected to its operand nodes. Numeric constants are wrapped as scalar values; invalid operands are rejected.

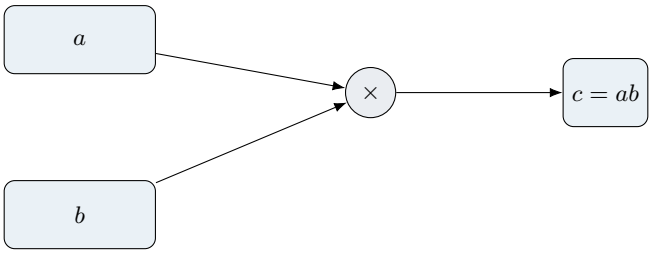


Figure 1. Forward graph for scalar multiplication. Reverse-mode gradients travel from c to a and b.

2.2 Local derivative rules

Operation	Forward definition	Local derivative
Addition	$z = a + b$	$\partial z/\partial a = 1,\ \partial z/\partial b = 1$
Multiplication	$z = ab$	$\partial z/\partial a = b,\ \partial z/\partial b = a$
Power	$z = a^n$ (constant n)	$\partial z/\partial a = na^{n-1}$
Hyperbolic tangent	$z = \tanh(a)$	$\partial z/\partial a = 1 - z^2$
Exponential	$z = e^a$	$\partial z/\partial a = z$
Division	$z = a/b = a\,b^{-1}$	Composition of multiplication and power

3 Reverse-mode backpropagation

For a scalar output L, every intermediate node receives an upstream gradient $\partial L/\partial z$. The chain rule contributes $\frac{\partial L}{\partial a} \mathrel{+}= \frac{\partial L}{\partial z}\frac{\partial z}{\partial a}$. Accumulation with `+=` is necessary whenever multiple paths converge on one node.

3.1 Topological ordering

Depth-first search visits each node's parents before appending the node to the topological list. A `visited` set prevents repeated visits in shared subgraphs. Backpropagation initializes the root gradient to 1 and calls local callbacks in *reverse* topological order. The implemented method clears gradients on the reachable graph before each full backward pass, making repeated calls deterministic; this differs from frameworks that accumulate across calls by default.

3.2 Diamond graph derivation

Let $a = 3$, $c = a + a = 2a$, and $L = c\,a = 2a^2$. The two paths from a to L contribute to the same derivative:

$$\frac{dL}{da} = \frac{d(2a^2)}{da} = 4a = 12.$$

The original course test verifies that `a.grad == 12.0`. Using assignment rather than accumulation would discard one or more contributions.

4 Neural network API

A neuron evaluates $z = \sum_{i=1}^{n} w_i x_i + b$ and returns $\tanh(z)$. A `Layer` groups n_{out} independent neurons; an `MLP` composes successive layers. Weights and biases are initialized independently from $\mathcal{U}(-1, 1)$, and `parameters()` exposes all learnable `Value` objects. `zero_grad()` clears parameter gradients.

For the course test architecture $3 \to 4 \to 4 \to 1$, the parameter count is

$$(3 \cdot 4 + 4) + (4 \cdot 4 + 4) + (4 \cdot 1 + 1) = \boxed{41}.$$

Input dimensionality and positive layer sizes are checked explicitly to avoid silent truncation by Python's `zip`.

5 Notebook experiment design

The supplied notebook defines a reproducible synthetic *two-moons* binary classification task, using NumPy for data generation and Matplotlib for plotting. The random seed is 42. It creates 80 samples (40 per class), with 60 training and 20 test samples in a stratified split. Input features are standardized using *training-set statistics only*, preventing test leakage. Targets are -1 and $+1$, matching the output range of `tanh`.

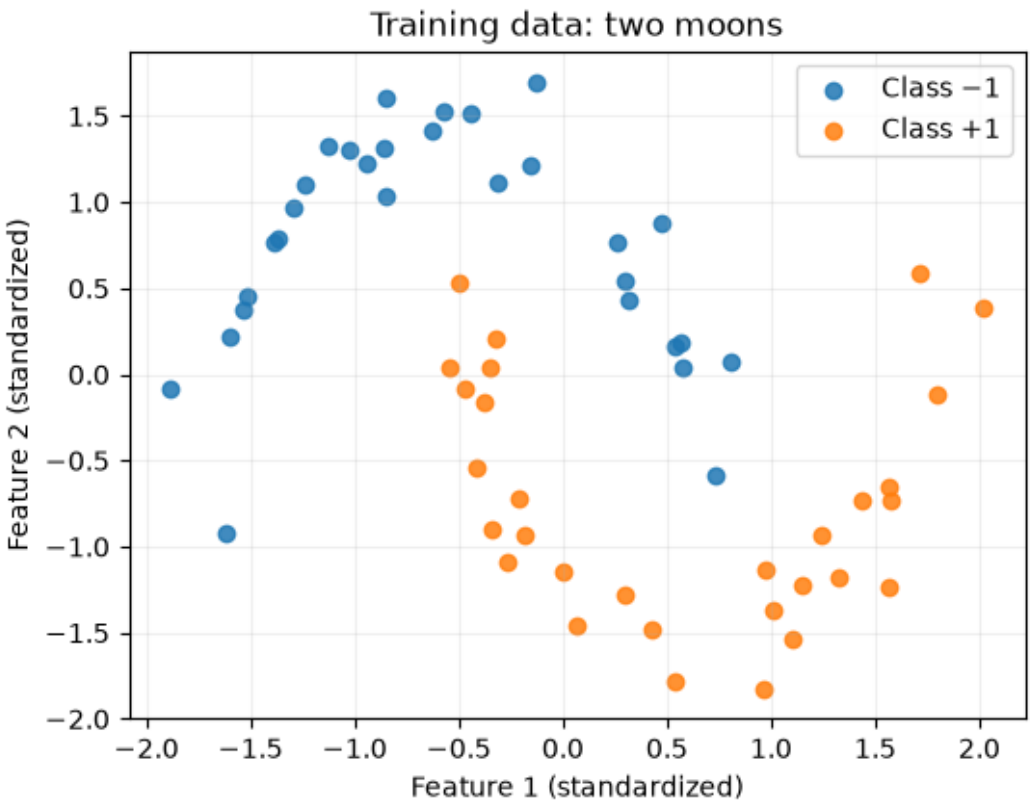


Figure 2. Executed notebook output: seeded two-moons training data.

The notebook architecture is $2 \to 8 \to 8 \to 1$ with $8(2+1) + 8(8+1) + 1(8+1) = \boxed{105}$ trainable parameters. Its objective is mean squared error:

$$L = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2, \qquad \theta \leftarrow \theta - \eta \nabla_\theta L.$$

The notebook specifies 180 epochs, full-batch gradient descent, and learning rate $\eta = 0.05$. Classification uses the sign of the output score; accuracy and loss are monitored, and a decision boundary is visualized.

6 Verification and results

Validation suite	Confirmed results
Original course tests	12 passed
Additional engine tests	13 passed
Additional neural network tests	9 passed
Total	**34 passed (0.19 s)**

The tests cover scalar arithmetic, shared-node gradient accumulation, topological ordering, input validation, full-network numerical gradients, and gradient-descent learning. A separate $2 \to 4 \to 1$ training sanity test reduced MSE from 0.135299 to 0.000017 over 300 epochs; this is distinct from the two-moons experiment.

6.1 Executed two-moons classification experiment

The newly supplied notebook contains saved execution outputs for all plots and reported metrics. Its $2 \to 8 \to 8 \to 1$ network trained for 180 epochs with full-batch gradient descent and learning rate 0.05.

Metric	Saved notebook result
Initial training MSE	1.255750
Final training MSE	0.109926
Training accuracy	98.3%
Test accuracy	100.0%
Training / test samples	60 / 20
Trainable parameters	105

The observed training-loss reduction is approximately 91.2%. Test accuracy is measured on only 20 synthetic samples and should not be interpreted as evidence of real-world generalization.

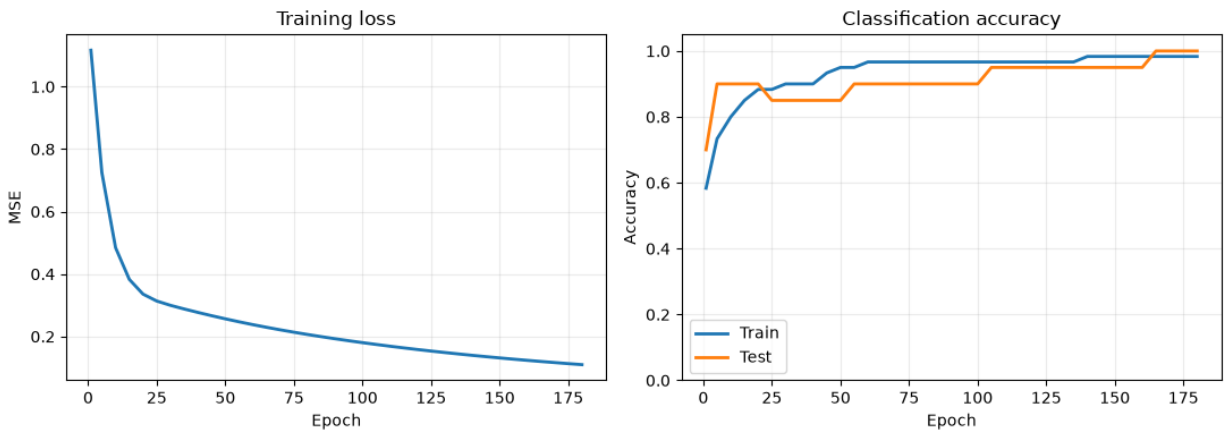


Figure 3. Actual notebook output: training loss and classification accuracy over 180 epochs.

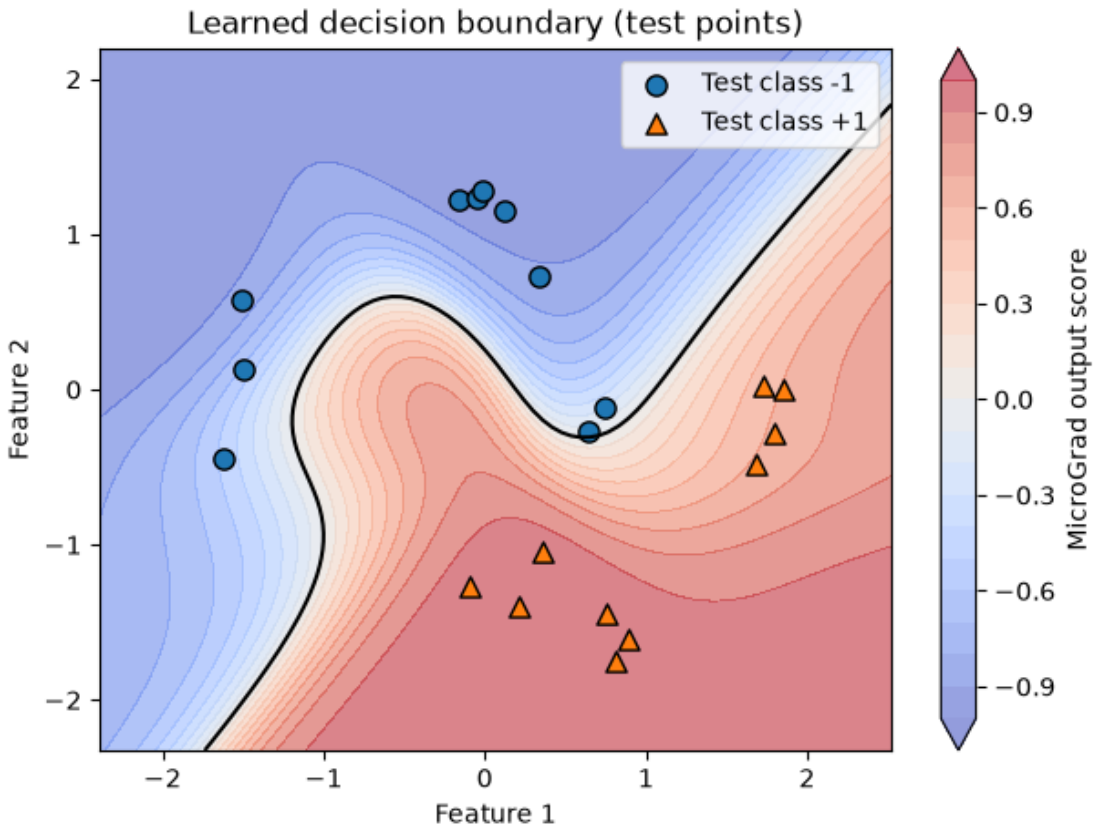


Figure 4. Actual notebook output: learned decision boundary and classified data points.

6.2 Numerical gradient checking

For a selected parameter θ, the notebook compares backpropagation to centered finite differences:

$$g_{\text{num}} \approx \frac{L(\theta + h) - L(\theta - h)}{2h}, \qquad h = 10^{-5}.$$

Parameter index	Absolute gradient error
0	1.18×10^{-10}
52	5.69×10^{-12}
104	3.34×10^{-11}

All three checks passed in the saved notebook execution. This verifies selected gradients against an independent numerical approximation; it does not exhaustively check every parameter.

7 Design decisions, limitations, and conclusion

Design choices. Scalar nodes keep the engine easy to inspect and mathematically transparent. Reverse-mode differentiation is efficient for a scalar loss with many parameters. Reusing arithmetic operators enables natural expression construction. Seeded initialization and a deterministic dataset make experiments repeatable.

Limitations. Scalar graph construction is slower and more memory-intensive than tensorized autodiff; the engine does not implement broadcasting, vectorized kernels, or general differentiable exponents. The selected notebook uses MSE for classification rather than cross-entropy, and the small synthetic dataset is illustrative rather than a production benchmark.

Conclusion. The implementation satisfies the visible course requirements for scalar autograd, topological backpropagation, and a basic neural network API. All 34 locally reported tests passed. The executed notebook now documents the training curves, decision boundary, evaluation metrics, and three numerical gradient checks.

References

Andrej Karpathy, *micrograd* (conceptual inspiration).

Ain Shams University, *Deep Learning Assignment 1: MicroGrad (Scalar Autograd Engine)*, course handout.

Student's executed `micrograd_demo.ipynb` (saved outputs) and reported local `pytest` output.